

Subject: PRF193 - Programming Fundamentals (with C/C++).

Topic 04: Airline Flight & Passenger Reservation

1. Objectives:

Design and implement a robust C++ application to manage software developers and their assigned projects. This project requires the application of advanced C++ concepts:

- **Object-Oriented Programming (OOP):** Encapsulation, Inheritance, and Polymorphism.
- **Standard Template Library (STL):** Effective use of containers (e.g., `std::vector`, `std::map`) and algorithms.
- **Memory Management:** Proper use of constructors, destructors, and smart pointers.
- **File Processing:** Persistent data storage using text or binary files.
- **Advanced Features:** Exception Handling for robust input validation.

2. Deliverables & Assessment

- **Group Size:** 4 students.
- **Submission Files:**
 - **Source Code:** Fully commented C++ files.
 - **Technical Report:** A PDF explaining the architecture and implementation (class diagram and flow-chart).
 - **Presentation:** Slides covering the core logic and features.
- **Duration:** 09 weeks (from 2nd to 10th).
- **Submission:** via LMS (as directed by the instructor).

3. Functional Requirements

A. Flight Management

Manage professional profiles with the following attributes:

- **Attributes:**
 - Flight ID (Format: FLTxxx, unique)

- Destination
- Departure Time
- Ticket Price (min. \$50)
- **Operations:**
 - Add flight schedules
 - Update flight schedules
 - Search by destination
 - Calculate total revenue per flight

B. Reservation Management

Manage flight bookings:

- **Attributes:**
 - Booking ID
 - Passenger ID (Format: PASxxx)
 - Full Name (min. 2 words)
 - Seat Class (Economy/Business)
- **Operations:**
 - Book a ticket (link to Flight ID)
 - Cancel reservation
 - Display passenger lists grouped by flight.

C. Data Analysis & UI

Search & Sort: Sort flights by Ticket Price (ascending) or Departure Time.

D. User Interface:

- A professional console-based menu system for easy navigation.
- Design a simple and intuitive text-based menu system.

E. File I/O:

- Load and save data from/to text or binary files.

4. Technical Requirements:

Requirement	Description
Modular Programming	Code must be organized into multiple header (.h) and implementation (.cpp) files.
Data Persistence	Automatically load data from file on startup and save upon exit.
Encapsulation	All class members must be private with appropriate getters, setters, and constructors.
Memory Safety	Use smart pointers (std::unique_ptr or std::shared_ptr) to prevent memory leaks.
Validation	Use try-catch blocks to handle invalid inputs, such as negative salaries or non-existent IDs.

-- End --.